Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Name and Signature of Student and Date
(Signature and Date in Handwritten)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session	**2025-26 (ODD)**	**Course Name**	**Operating System Lab**
Semester	**5**	**Course Code**	**23IOT1504**
Roll No	**64**	**Name of Student**	**Sheikh Wasimuddin**

Practical Number	**06**
Course Outcome	1. Understand Computer System Configuration and Simulate system resources efficiently using Linux Commands (CO1) 2. Analyse operating system functionalities utilizing system calls, thread programming and process scheduling algorithms (CO2) 3. Apply Synchronization primitives to implement a Deadlock-free solution(CO3) 4. Simulate Disk scheduling, Memory allocation, File allocation, page replacement algorithms (CO4)
Aim	Simulate the Page Replacement Algorithms(Any Two) A. First in First Out(FIFO) B. Least Recently Used(LRU) C. Optimal
Theory	Page replacement is a memory management technique used by operating systems when a program needs a page that is not currently available in the physical memory (RAM). Since physical memory is limited in size, not all pages of a process can be stored at the same time. When a page fault occurs and the memory is already full, the operating system must decide which existing page to remove to make space for the required one. This decision process is known as a Page Replacement Algorithm. A. First in First Out(FIFO) B. Least Recently Used(LRU) C. Optimal ❖ **First in First Out(FIFO):** The FIFO page replacement algorithm is based on the idea that the page that has been loaded into memory earliest should be removed first when a new page needs space. It uses a queue structure where pages are inserted at the rear and replaced from the front. The operating system does not consider how frequently or recently a page has been used—only the order in which pages entered the memory. Although FIFO is simple to implement, a major drawback is that it may lead to more page faults even when the number of frames is increased, known as Belady's Anomaly.

	This makes FIFO less effective for workloads where older pages are still actively required. ❖ **Least Recently Used (LRU) Page Replacement:** The Least Recently Used algorithm removes the page that has not been used for the longest time. It assumes that a page accessed recently is more likely to be used again soon. To apply LRU, the system must continuously track page usage either through timestamps or stack-like structures. When a page fault occurs and memory is full, the page with the oldest reference (least recently used) is replaced. LRU generally performs better than FIFO because it considers actual usage behavior, but it comes with higher implementation complexity and overhead due to tracking the order of access. ❖ **Optimal Page Replacement:** The Optimal page replacement algorithm replaces the page that will not be used for the longest period in the future. It works by predicting future page references and choosing the best possible page to remove. This algorithm provides the minimum possible number of page faults, serving as a theoretical benchmark for comparison with other algorithms. However, since future memory references are not realistically known in advance, Optimal is not used in actual systems. Instead, it is mainly used in research, evaluation, and academic simulations to measure the efficiency of practical algorithms like FIFO and LRU.

Procedure and Execution	**A. FIFO:**
	Code:
	//sheikh wasimuddin 23071044 cseiot-64
	#include <stdio.h>
	#include <stdbool.h>
	int main() {
	int n, framesCount, page, faults = 0;
	printf("Enter number of pages: ");
	scanf("%d", &n);
	int pages[n];
	printf("Enter page reference string: ");
	for (int i = 0; i < n; i++) {
	scanf("%d", &pages[i]);
	}
	printf("Enter number of frames: ");
	scanf("%d", &framesCount);
	int frames[framesCount];
	int current = 0;
	for (int i = 0; i < framesCount; i++) {

```
        frames[i] = -1;
    }

    for (int i = 0; i < n; i++) {
        bool found = false;

        for (int j = 0; j < framesCount; j++) {
            if (frames[j] == pages[i]) {
                found = true;
                break;
            }
        }

        if (!found) {
            faults++;
            frames[current] = pages[i];
            current = (current + 1) % framesCount;
        }

        printf("Frame: ");
        for (int j = 0; j < framesCount; j++) {
            if (frames[j] != -1)
                printf("%d ", frames[j]);
            else
                printf("- ");
        }
        printf("\n");
    }

    printf("Total Page Faults: %d\n", faults);

    return 0;
}
```

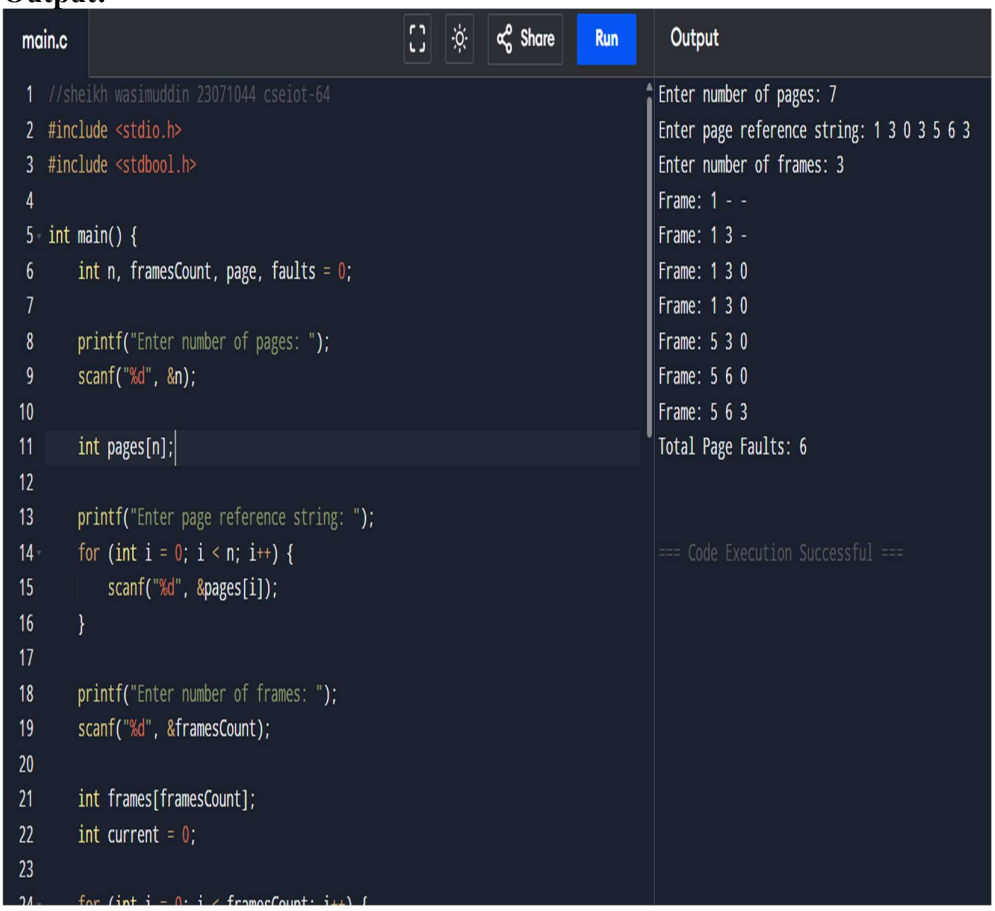
B. **LRU:**

```
//sheikh wasimuddin 23071044 cseiot-64
#include <stdio.h>
#include <stdbool.h>
int main() {
    int n, framesCount, faults = 0;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    int pages[n];
    printf("Enter page reference string: ");
```

```
for (int i = 0; i < n; i++) {
    scanf("%d", &pages[i]);
}
printf("Enter number of frames: ");
scanf("%d", &framesCount);
int frames[framesCount];
int recent[framesCount];
for (int i = 0; i < framesCount; i++) {
    frames[i] = -1;
    recent[i] = 0;
}
for (int i = 0; i < n; i++) {
    int currentPage = pages[i];
    bool found = false;
    for (int j = 0; j < framesCount; j++) {
        if (frames[j] == currentPage) {
            found = true;
            recent[j] = i;
            break;
        }
    }
    if (!found) {
        faults++;
        int replaceIndex = 0;
        for (int j = 0; j < framesCount; j++) {
            if (frames[j] == -1) {
                replaceIndex = j;
                break;
            }
            if (recent[j] < recent[replaceIndex]) {
                replaceIndex = j;
            }
        }
        frames[replaceIndex] = currentPage;
        recent[replaceIndex] = i;
    }
    printf("Frame: ");
    for (int j = 0; j < framesCount; j++) {
        if (frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
    printf("\n");
```

```
        }
        printf("Total Page Faults: %d\n", faults);
        return 0;
    }
```

Output:

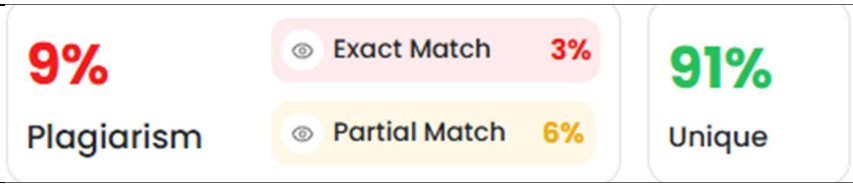


<table>
<tr><td></td><td>

```c
//sheikh wasimuddin 23071044 cseiot-64
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, framesCount, faults = 0;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    int pages[n];

    printf("Enter page reference string: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &framesCount);

    int frames[framesCount];
    int recent[framesCount];

    for (int i = 0; i < framesCount; i++) {
```

```
Output
Enter number of pages: 8
Enter page reference string: 1 2 3 2 1 4 5 2
Enter number of frames: 3
Frame: 1 - -
Frame: 1 2 -
Frame: 1 2 3
Frame: 1 2 3
Frame: 1 2 3
Frame: 1 2 4
Frame: 1 5 4
Frame: 2 5 4
Total Page Faults: 6

=== Code Execution Successful ===
```

</td></tr>
<tr><td>Output Analysis</td><td>From the observed results of both FIFO and LRU page replacement algorithms, it is clear that the behavior of page fault generation varies based on the way each algorithm selects a page for replacement. In FIFO, pages are removed in the same order they entered memory, without considering their future or recent usage. This leads to more page faults because actively used pages might get replaced simply due to their age in memory. On the other hand, LRU uses the reference history of pages and removes the one that has not been used for the longest time. This reduces page faults by keeping frequently accessed pages in memory for longer. Therefore, the analysis shows that **LRU performs more efficiently than FIFO** in most cases, although FIFO is simpler to implement.</td></tr>
<tr><td>Conclusion</td><td>The implementation and simulation of FIFO and LRU page replacement algorithms demonstrate how memory management strategies directly affect system performance. FIFO is simple and easy to implement, but it may lead to more page faults due to its inability to consider recent page usage. LRU performs better as it keeps track of page reference patterns and replaces the least recently used page, reducing unnecessary faults. From the output results, it is concluded that **LRU provides higher efficiency and improved memory utilization compared to FIFO**, although it requires more processing overhead. Overall, choosing the right</td></tr>
</table>

	page replacement algorithm is important for optimizing operating system performance.
Plag Report (Similarity index < 12%)	
Date	06/10/2025